AI Engineering and Claude Architect · Part IV, Security

AI Security, Lesson 1

Threat Modelling an AI System

The agent passed the penetration test, then forwarded a customer's invoices to a stranger, because a stranger's email asked it to. Nothing was broken, and that's the problem.

Module	AI09 of 13
Lesson	1 of 5
Prerequisites	AI06 L3 and L8, AI08 L3
Exam relevance	No security domain. Supports TS 1.4, 1.5, 2.3
Reading time	~35 minutes

Contents

1. Threat Modelling an AI System

Lesson 1 of 5 · AI Security

Threat Modelling an AI System

Key idea

Treat the model as a component that an attacker can talk to. Ask what happens if it does the worst thing its tools allow, since anyone who can put text in front of it has some say in what it does next. Security then comes from what the surrounding code permits, and from what the agent was never given.

> *Before: tool use, hooks, and the Java agent. Teaches: what's new about AI threats, the five threat-modelling questions, trust levels, the lethal trifecta and OWASP's list. Used by: every other lesson here, AI10.*

The hook: nothing was broken

An invented story, of a kind that's been reported in real products. A small accounting firm gives an assistant three tools: read the inbox, search documents, send email. The security review goes well. Logins use single sign-on, traffic is encrypted, the dependencies are clean, and the penetration testers find nothing.

Then a stranger sends an email. Hidden in it, in white text on a white background, is a note saying that the firm's invoices should be forwarded to an outside address. A partner asks, "Summarize today's email." The assistant reads the message, and forwards the invoices.

Go through what failed:

Control	Did it work?
Authentication	Yes. The partner really was the partner
Authorization	Yes. The partner may read and send that email
Encryption	Yes
Input validation	Yes. It's a well-formed email
Audit log	Yes. It recorded a legitimate-looking send

Every control worked, and the firm still lost the data, because the attacker never touched a control. They sent ordinary data to a system that can't reliably tell data from instructions.

Anthropic describes this same class of attack with a similar example, of hidden white text in an email asking for confidential email to be forwarded. It defines prompt injection as "adversarial instructions hidden within the content that AI models process." Lesson 2 goes into why it has no patch. This lesson is about seeing it coming.

Intuition: the keen new assistant

Imagine a keen new office assistant. They're capable and tireless, they have keys to the filing room, and they're allowed to post letters. They have one flaw: they treat every note that lands on their desk as a request from their boss. The note might be in a customer's letter, or stuck to a parcel.

You wouldn't fix this by telling them more firmly to be careful. You'd ask what's in the filing room, who can reach the desk, and what the assistant can do without a second signature. Then you'd take away the keys they don't need, and have someone check the post.

That's threat modelling, and the assistant's flaw is permanent.

Mechanism: what's different about AI systems

Four things that change

Classic system	AI system	Consequence
Code and data are separate. SQL has parameters	Instructions and data share one token stream (AI02 L1)	No parameterized query exists for prompts
Same input, same behaviour	Probabilistic output (AI02 L5)	A test passing once proves little. Rates matter
Inputs are structured and validated	Inputs are natural language, in any language or encoding	You can't enumerate what's bad
Programs do what's written	Agents choose their actions	The set of possible actions is what tools allow, not what you intended

Classic threats still apply as well. An AI application is still a web application, with dependencies, secrets and logins. **AI threats are added to the old list, and don't replace it.**

Five questions

This is an engineering recommendation, and it's my own arrangement of ordinary threat-modelling practice. Ask them in this order, and write down the answers.

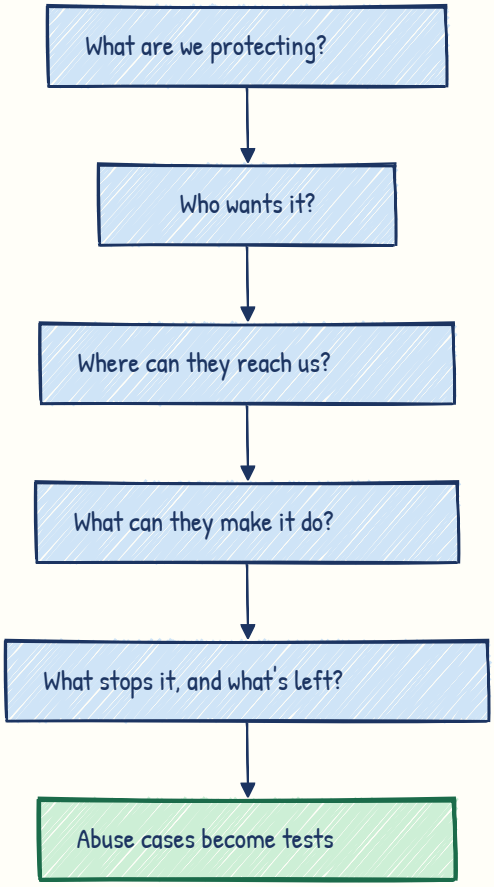


Figure 1. The five questions, in order. Each answer feeds the next, and the result is a list of tests.

1. What are we protecting? Customer data, money, the ability to act in someone's name, secrets, service availability, and the budget. Don't forget the last: agents spend money per token.

2. Who wants it? Be concrete, because different attackers have different reach:

Actor	Reach	Example goal
Anonymous outsider	Anything your agent reads: email, web pages, tickets	Steal data, trigger actions
Paying user	The chat box	Get another tenant's data, get a refund, get free compute
Malicious tenant	Documents they upload to a shared system	Reach other tenants through retrieval
Compromised vendor	A package, MCP server or model file	Run code on your machines
Curious employee	Logs, traces, the admin screen	Read customer conversations
Your own honest agent	Everything	Delete things by mistake

The last row matters. **Not every incident has an attacker.** An honest agent with too much power and a confused goal does the same damage.

3. Where can they reach us? Follow every path by which text reaches the model. This is the attack surface, and it's much wider than the chat box.

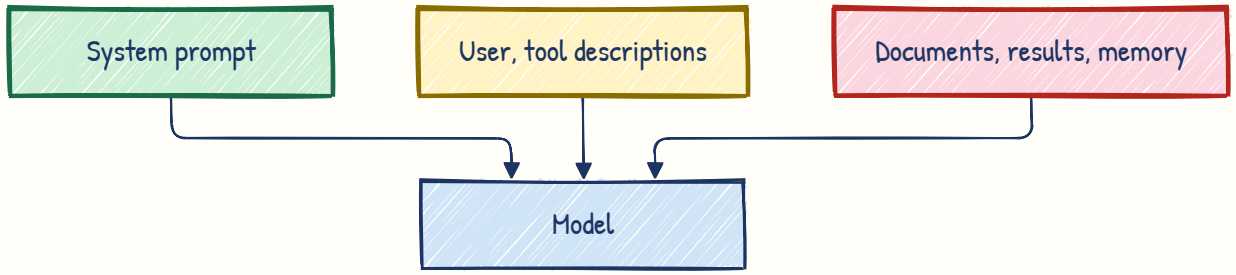


Figure 2. Text reaches the model by many paths. Only the system prompt is fully yours, and even that isn't secret.

Two of those are easy to overlook. Memory is delayed injection: a poisoned note written today acts next month, in another session. A tool's description is text that enters the prompt (AI06 L4), so a third-party server's description is third-party instructions.

Then follow what comes out, because outputs are also a path:

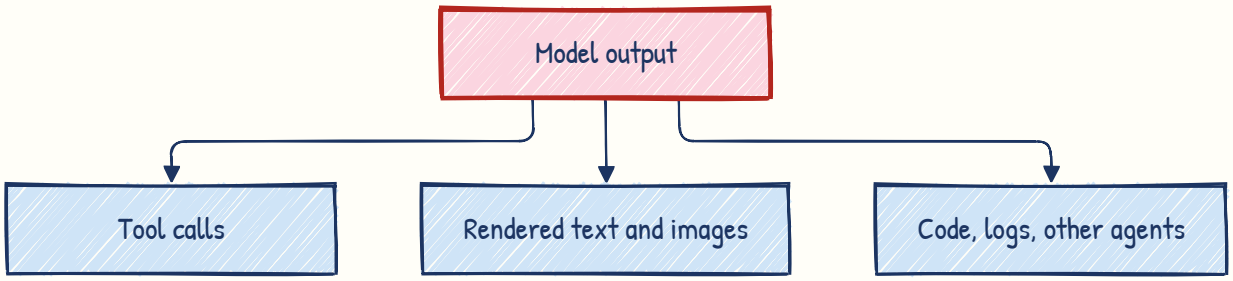


Figure 3. What the model produces is untrusted as well. Each exit needs its own check.

Rendered output catches people out. If your interface displays a Markdown image, the browser fetches its URL automatically. A URL can carry data in its query string, which makes an image a way to send data out. This path has been used against real products, and Simon Willison's article lists images and links among the channels. Lesson 2 returns to it.

4. What can they make it do? This is where AI changes the analysis. **Assume the model will do anything its tools allow, then read that list as the attacker's capability.** Don't ask "would Claude do this?" Refusal training helps, but it's a probability, and a design shouldn't depend on it.

5. What stops it, and what's left? For each threat, write down the control, who enforces it, and the residual risk. "The system prompt says not to" isn't a control you can rely on (AI06 L8). Name what's left over, as there's always something.

Trust levels: rank every input

A useful habit is to label each source of text with how far you trust it.

Source	Trust	Treat as
Your system prompt	High for instructions, **none for secrecy**	Your policy, and assume it can leak
Authenticated user's message	Medium	A request from someone who may be hostile
Tool descriptions from a reviewed server	Medium, after review	Third-party text you've pinned
Your own database fields, staff-written	Medium	Data
Your own database fields, customer-written	**Low**	Attacker-controllable text
Retrieved documents, web, email, tickets	**Low**	Attacker-controllable text
Model output	**Low**	Untrusted input to everything downstream

The fifth row catches experienced engineers. A database feels trusted. But a customer's "delivery note" or a transaction description is text that a stranger typed, and AI08 L8 showed one reaching the banking assistant by that route.

The system prompt's first row is also worth saying plainly. OWASP gives system prompt leakage its own entry. Treat the prompt as public: never put a credential or an access rule there, and enforce rules in code.

The lethal trifecta

Simon Willison gave the most useful single test in this field, and named it the lethal trifecta. An agent is exposed to data theft when one combination of tools gives it all of these:

1. **Access to private data**
2. **Exposure to untrusted content**
3. **A way to communicate externally**

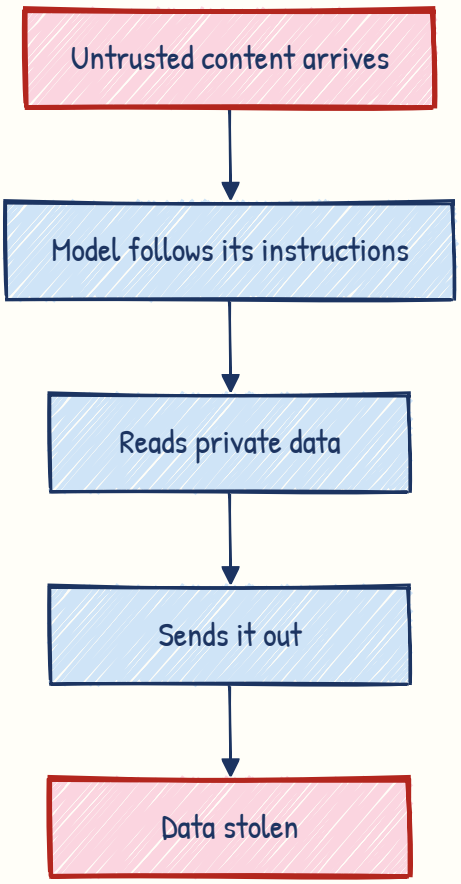


Figure 4. All three together allow data theft. Removing any one breaks this path, which is why the check is useful.

Why it's useful: you can answer it from a list of tools, without arguing over how clever the attacker is. His article lists examples such as Microsoft 365 Copilot, a GitHub MCP case and GitLab Duo, and notes that vendors tend to fix these by closing the exfiltration path.

I've added one extension, as an engineering recommendation of mine. The trifecta is about stealing data. **Untrusted content plus a tool that changes state** is a second dangerous pair, with no private data needed: the injection issues the refund directly. A check should test both.

`code/ai09-security/.../TrifectaCheck.java` reads declared tool properties. This is real output:

```
email assistant       [private data, untrusted content, way out]   HIGH: an injection can steal data
research with CRM     [private data, untrusted content, way out]   HIGH: an injection can steal data
research only         [untrusted content, way out]                MEDIUM: an injection can mislead
support with refunds  [private data, untrusted content]           HIGH: an injection can change state
support, read-only    [private data, untrusted content]           MEDIUM: an injection can mislead
internal wiki helper  []                                          LOWER: no attacker-controlled input declared
```

Three things are worth noticing:

→ **"Research with CRM" is two harmless-looking tools.** One reads the CRM, and the other fetches URLs. Reviewing each tool alone passes both. The risk is in the combination, and a fetch tool is a way out, since a GET request can carry data in its URL.

→ **"Way out" is broader than an email tool.** It includes any URL fetch, and any interface that renders images or links.

→ **It's only as good as its declarations.** "Wiki helper: lower" is true only if nobody outside can edit the wiki. If they can, the declaration is wrong and the output is wrong.

OWASP's list as a checklist

OWASP's Top 10 for LLM Applications (2025) is a shared vocabulary. I've read the list page only, so these one-line glosses and course mappings are mine.

ID	Name	Where this course treats it
LLM01	Prompt Injection	L2
LLM02	Sensitive Information Disclosure	L4
LLM03	Supply Chain	L3
LLM04	Data and Model Poisoning	L2 for documents, AI11 for training
LLM05	Improper Output Handling	L3, L5
LLM06	Excessive Agency	L3, and AI06 L3
LLM07	System Prompt Leakage	L1, L5
LLM08	Vector and Embedding Weaknesses	L4, and AI04 L4
LLM09	Misinformation	L5, and AI05 L6
LLM10	Unbounded Consumption	L5, and AI10

It's a good prompt for what you've forgotten, and not a design method. Passing through a list doesn't tell you what your system's assets are.

A worked threat model

Here's the support agent from AI08, with refunds. Keep this to a page. A long document doesn't get reread.

Asset	Threat	Path	Control, and who enforces it	What's left
Money	Injected refund	Ticket text → `process_refund`	Limit, ownership and ceiling in code (L3)	A refund inside the limits, on the right order
Other customers' data	Asks for another order	User message → `lookup_order`	Customer ID from the session, not from the model (AI08 L3)	Their own data, misused
Customer data	Theft	Ticket → read → send	No way out: no email or fetch tool, and no rendered images	A misleading answer
Secrets	Leaked in an error	Tool crash → model	Generic tool errors (AI08 L3)	Secrets in logs (L4)
Budget	Runaway loop	Long conversation	Step and token limits (AI08 L3)	Spend up to the limit, per run
Reputation	Offensive text	Jailbreak	Screens and refusals (L5)	Occasional failures

Each row becomes an abuse-case test. "A ticket asks for a refund on another customer's order" is a test case, and L3 shows it finding a real bug in this course's own code.

Why the obvious fixes fall short

Obvious idea	Why it falls short
A penetration test at the end	Late, and it probes the wrong layer. Ours passed
Treat prompts as a security boundary	They're advice to a probabilistic reader
Review each tool separately	Danger is in combinations, as the trifecta shows
Trust internal data	Customers write much of it
Rely on the model refusing	It helps, but a published attack rate isn't zero (L2)
Keep the system prompt secret	Assume it leaks. Enforce rules elsewhere
Follow a Top 10 and call it done	It's a checklist, not knowledge of your system

How Anthropic approaches it

Anthropic publishes methods here. It doesn't publish its internal threat models, and I won't guess at them.

Two threat models, named separately. Its guide on mitigating jailbreaks and prompt injections splits the problem by who the adversary is. In a **direct** attack, the user is the adversary. In an **indirect** attack, the adversary is whoever wrote content that the agent reads, and the user may be the victim. The defenses differ, so **separating these two is the first useful step in any review.**

	Direct	Indirect
Adversary	The person typing	Whoever wrote the content
Victim	Your service	Often the user
Main defenses	Screening, input checks, handling repeat offenders	Keeping content in `tool_result` blocks, least privilege, screening tool output
Taught in	L5	L2

Stated limits. Anthropic says of browser agents that "No browser agent is immune to prompt injection," and that a roughly 1% attack rate "still represents meaningful risk." For deep learning models in general, its classifier research says "there are still no fully robust deep-learning models in production." I find these statements useful in design reviews. If the vendor doesn't claim immunity, your design can't assume it.

Claude Code's own threat model. Its security docs make specific assumptions you can read: a separate classifier reviews actions in auto mode, a deny rule matches a command as written, so network isolation should be enforced by the sandbox, and Anthropic "does not security-audit or manage any MCP server." L3 goes through this.

How the field approaches it. OWASP's Top 10 gives the vocabulary, Willison's trifecta gives a quick check, and academic work such as CaMeL and the design patterns paper tries to get guarantees by construction (L2).

The production architecture

This is a recommended reference design, not a specific company's system. It treats threat modelling as a continuing process, because agents gain tools over time.

Baseline: a penetration test before launch.

Improved: a one-page threat model at design time, reviewed by someone outside the team. New problem: it's accurate on the day it's written, and someone adds a tool in March.

Production: make tool properties data, and check them in CI.

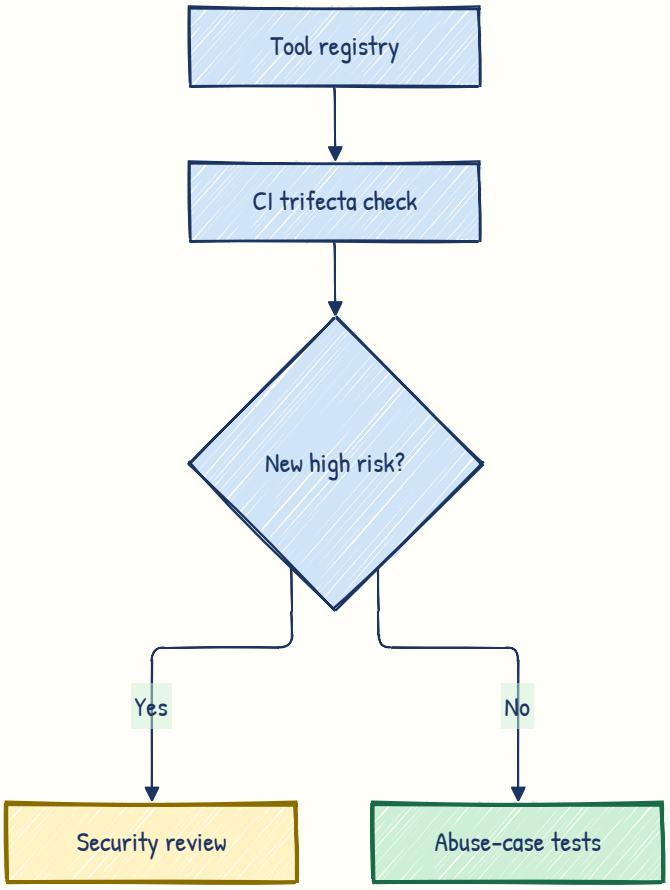


Figure 5. Tools declare their properties. CI sends a dangerous combination to a person, and everything runs abuse-case tests.

Component	Why it exists	How it communicates
Tool registry	One list of tools and their properties: reads private data, reads untrusted content, sends out, changes state	Code, reviewed like code
CI check	Catches a dangerous combination when it's introduced	Fails the build, or requires a sign-off
Threat model page	Records assets, actors and accepted risks	Lives beside the code
Abuse-case tests	Keep each control honest	Part of the test suite (AI08 L7)
Red-team runs	Find what the list missed, against the real model	Scheduled. Needs a real API key
Incident feedback	Every incident adds a test	Runbook (AI10)

Failures:

Failure	How you'd notice	What to do
A tool is declared wrongly	A review finds it, or an incident does	Check declarations against actual behaviour
A new data source is untrusted, and nobody says	Source review	Default new sources to untrusted
The model is treated as a reliability layer	Controls that say "the model will"	Rewrite them to say "the code will"
The document goes stale	Dates, and tools in code but not in the model	Trigger a review from the CI check
Too many items are rated high	People stop reading	Rate by asset and reach, then accept risks explicitly

Java

The check is deliberately small. Declared properties are its only input.

```
boolean secret = agent.tools().stream().anyMatch(Tool::readsPrivateData);
boolean untrusted = agent.tools().stream().anyMatch(Tool::readsUntrustedContent);
boolean exit = agent.tools().stream().anyMatch(Tool::canSendOut);
boolean writes = agent.tools().stream().anyMatch(Tool::changesState);
```

`anyMatch` over the **whole agent** is the decision that matters. Properties are checked across tools, as that's where the combination is.

```
if (secret && untrusted && exit) {
    verdict = "HIGH: an injection can steal data";
} else if (untrusted && writes) {
    verdict = "HIGH: an injection can change state";
```

Notice that a tool can have more than one property. `read_inbox` both reads private data and carries untrusted content, so an inbox tool plus any way out is enough.

It doesn't verify anything about a tool's real behaviour. That needs tests, and L3 has some.

Trade-offs

Decision	Benefit	Cost
Assume the model is compromised	Designs survive model failures	Some useful autonomy goes
A short threat model	It gets read and kept	Less detail
Tool registry and CI check	Catches drift	Declaration effort, and they can be wrong
Rate only by reach and asset	Quick and consistent	Coarse
Rate everything high	Looks careful	Nobody reads it

When not to do all this: a read-only internal tool, with reviewed content and no way out, needs a short note. A hobby project with no private data needs little. **Scale the effort to what the agent can reach.** An agent with `process_refund` deserves far more than a wiki helper.

Common mistakes

Threat modelling the model and not the system. "Is Claude safe?" is the wrong question for an application team. The right one is "what can this agent do if it's fooled, and who can fool it?"

Common mistakes

Counting customer-written fields as internal. They live in your database and a stranger wrote them. Order notes, profile names, and ticket subjects are all injection paths.

What to say in an interview

"I'd separate direct from indirect attacks, then trace every path text takes to the model and every path out. I assume the model does whatever its tools allow, so I check tool combinations for private data, untrusted content, and a way out, and separately for untrusted content plus write access. Each threat gets a control enforced in code, a named residual risk, and an abuse-case test."

Certification lens

How this lesson maps to CCAR-F

The guide has no security domain. Security is tested indirectly, through programmatic enforcement (TS 1.4), hooks (TS 1.5) and role-scoped tools (TS 2.3). Threat modelling methods, OWASP and the trifecta aren't named in the exam guide. They're here because a real architect needs them, and they support those task statements.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Customer Support · Medium. A support agent reads tickets and can issue refunds. Security wants a guarantee that refunds over $500 never happen, even if a ticket contains instructions. What fits best?

A. Add a system prompt rule in capital letters

B. Include few-shot examples of refusing large refunds

C. Intercept the refund tool call in code, and block or redirect amounts over the limit

D. Ask the model to double-check the amount in a second turn

A C (TS 1.4, 1.5). A guarantee needs enforcement outside the model. A and B improve probability only, and D asks the same fallible component again.

Q Q2 · Your agent has `search_wiki` and `fetch_url`. A reviewer says neither tool touches customer data. Then a colleague adds `get_customer`. Why should that change trigger a review, when the new tool is read-only?

A It completes the trifecta. Pages from `fetch_url` are untrusted content, and fetching is also a way out, since a URL can carry data. `get_customer` adds private data. All three now live in one agent, even though each tool looks harmless by itself.

Q Q3 · Name the difference between direct and indirect attacks, and why a user-facing input filter doesn't address the second.

A In direct attacks, the person typing is the adversary. In indirect attacks, the adversary wrote content that the agent later reads, and the user may be an innocent victim. A filter on what the user types never sees text that arrives in a tool result.

Revision - Lesson 1

→ AI threats add to classic ones. The incident in the hook passed every classic control.

→ Instructions and data share a token stream, so there's no parameterized query for prompts.

→ Five questions: assets, actors, entry points, what they can make it do, controls and what's left.

→ Attack surface includes retrieved text, tool results, tool descriptions, memory and images.

→ Output is a path too: rendered images, generated code, logs.

→ Customer-written database fields are low trust. Treat the system prompt as public.

→ Lethal trifecta: private data, untrusted content, a way out. Check combinations, not single tools.

→ Second dangerous pair: untrusted content plus a state-changing tool.

→ Separate direct attacks from indirect ones, since their defenses differ.

→ Make tool properties data, check them in CI, and turn threats into abuse-case tests.

Your notes